

Day 42 : useEffect Hook

1. Introduction to useEffect

What is useEffect?

`useEffect` is a React Hook that lets you perform **side effects** in your functional components.

Side effects are operations that affect things outside your component:

- Fetching data from APIs
 - Setting up timers (setInterval, setTimeout)
 - Subscribing to external services
 - Manually updating the DOM
 - Adding event listeners
-

Why Do We Need useEffect?

React components are functions that:

1. Run
2. Return JSX
3. React updates the screen

This happens **synchronously** - React needs JSX immediately.

Problem: What if you need to do something that takes time (like fetching data)?

You can't pause the component function and wait!

Solution: `useEffect` runs your code **AFTER** the component renders.

2. The Problem useEffect Solves

React's Rendering Model

Component function runs → Returns JSX → React updates screen

This is **synchronous** and **fast**.

What Happens Without useEffect?

Example: Fetching Data Without useEffect

```
function App() {  
  const [data, setData] = useState(null);  
  
  // ❌ BAD! Don't do this  
  fetch('https://api.example.com/users')  
    .then(res => res.json())  
    .then(result => setData(result));  
  
  return <div>{data ? data.name : 'Loading...'}</div>;  
}
```

What happens:

1. Component renders → fetch runs
2. Fetch completes → setData called
3. State changes → Component re-renders
4. Component renders → fetch runs AGAIN
5. Fetch completes → setData called
6. State changes → Component re-renders
7. Infinite loop! 🔄💥

Why? Every render calls `fetch`, which eventually calls `setData`, which triggers another render!

3. Basic Syntax and Structure

useEffect Basic Structure

```
import { useEffect } from 'react';

useEffect(() => {
  // 1. Effect function - Your side effect code here

  return () => {
    // 2. Cleanup function (optional) - Clean up code here
  };
}, [dependencies]); // 3. Dependency array - Controls when effect runs
```

Three parts:

1. **Effect Function** - The code you want to run
2. **Cleanup Function** (optional) - Code to clean up (returned from effect)
3. **Dependency Array** - Tells React when to run the effect

When Does useEffect Run?

Timeline:

1. Component renders (JSX returned)
2. Screen updates (user sees changes)
3. THEN useEffect runs (after everything)

Key Point: useEffect runs **AFTER** the render, not during it. This prevents blocking the UI.

4. Using fetch with useEffect

The Correct Way to Fetch Data

```
import { useState, useEffect } from 'react';
```

```
function App() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('https://api.example.com/users')
      .then(res => res.json())
      .then(result => setData(result));
  }, []); // ← Empty array = run once

  return <div>{data ? data.name : 'Loading...'}</div>;
}
```

What happens:

First Render:

1. Component renders with data = null
2. Returns "Loading..."
3. Screen shows "Loading..."
4. AFTER render: useEffect runs
5. Fetch starts (happens in background)

When fetch completes:

6. setData called with result
7. Component re-renders with data
8. useEffect sees [] → "Already ran, skip"
9. Returns data.name
10. Screen shows the data
11. Done! 

The empty array  is KEY - it prevents the infinite loop!

Step-by-Step Breakdown

Step 1: Initial Setup

```
const [data, setData] = useState(null);
```

- State starts as `null`
- We'll store fetched data here

Step 2: useEffect with Fetch

```
useEffect(() => {  
  fetch('https://api.example.com/users')  
    .then(res => res.json())  
    .then(result => setData(result));  
}, []);
```

- Effect runs **once** after first render (because of `[]`)
- Fetches data
- Updates state when data arrives

Step 3: Display Data

```
return <div>{data ? data.name : 'Loading...'}</div>;
```

- First render: `data` is `null` → Shows "Loading..."
- After fetch: `data` has value → Shows the name

Using async/await with useEffect

✗ Wrong Way - Can't make effect function async:

```
useEffect(async () => { // ✗ Error!  
  const response = await fetch('/api/data');  
  const result = await response.json();
```

```
    setData(result);  
  }, []);
```

Why? `useEffect` expects either nothing or a cleanup function to be returned. But `async` functions return Promises!

✅ **Correct Way - Create async function inside:**

```
useEffect(() => {  
  async function fetchData() {  
    const response = await fetch('/api/data');  
    const result = await response.json();  
    setData(result);  
  }  
  
  fetchData(); // Call it immediately  
}, []);
```

✅ **Alternative - IIFE (Immediately Invoked Function Expression):**

```
useEffect(() => {  
  (async () => {  
    const response = await fetch('/api/data');  
    const result = await response.json();  
    setData(result);  
  })();  
}, []);
```

Complete Fetch Example with Loading and Error States

```
function UserList() {  
  const [users, setUsers] = useState([]);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);
```

```

useEffect(() => {
  async function fetchUsers() {
    try {
      setLoading(true);
      const response = await fetch('https://api.github.com/users');

      if (!response.ok) {
        throw new Error('Failed to fetch users');
      }

      const data = await response.json();
      setUsers(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  fetchUsers();
}, []);

if (loading) return <div>Loading...</div>;
if (error) return <div>Error: {error}</div>;

return (
  <ul>
    {users.map(user => (
      <li key={user.id}>{user.login}</li>
    ))}
  </ul>
);
}

```

5. Using setInterval with useEffect

Why Timers Need useEffect

✗ Without useEffect - Timers Pile Up:

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  // BAD! Don't do this  
  setInterval(() => {  
    console.log('Tick');  
  }, 1000);  
  
  return (  
    <div>  
      <p>{count}</p>  
      <button onClick={() => setCount(count + 1)}>Click</button>  
    </div>  
  );  
}
```

What happens:

First render:

→ Interval-1 created (ticks every second)

You click button:

→ Component re-renders

→ Interval-2 created (ticks every second)

→ Now TWO intervals running!

You click again:

→ Component re-renders

→ Interval-3 created

→ Now THREE intervals running!

Result: Timers keep piling up! Memory leak! 💣

✅ Correct Way - With useEffect and Cleanup

```
function Clock() {  
  const [time, setTime] = useState(new Date().toLocaleTimeString());  
  
  useEffect(() => {  
    const intervalId = setInterval(() => {  
      setTime(new Date().toLocaleTimeString());  
    }, 1000);  
  
    // Cleanup function  
    return () => {  
      clearInterval(intervalId);  
    };  
  }, []); // Run once  
  
  return <div>Time: {time}</div>;  
}
```

What happens:

Component mounts:

1. Effect runs
2. Interval created (stored in intervalId)
3. Cleanup function returned to React
4. Clock ticks every second

Every second:

- setTime called
- Component re-renders
- useEffect sees [] → "Already ran, skip"
- Interval keeps running (not recreated)

Component unmounts (removed from screen):

1. React calls cleanup function
2. `clearInterval(intervalId)` stops the interval
3. No memory leak! 

Step-by-Step: Understanding the Timer

Step 1: Store Interval ID

```
const intervalId = setInterval(() => {  
  setTime(new Date().toLocaleTimeString());  
}, 1000);
```

- `setInterval` returns an ID
- We need this ID to stop the interval later

Step 2: Return Cleanup Function

```
return () => {  
  clearInterval(intervalId);  
};
```

- This function will be called when component unmounts
- It uses the stored `intervalId` to stop the interval

Step 3: Empty Dependency Array

```
, []);
```

- `[]` means "run once on mount"
- Effect won't run again on re-renders

- But the cleanup WILL run when component unmounts

setTimeout Example

```
function Notification() {  
  const [show, setShow] = useState(true);  
  
  useEffect(() => {  
    const timeoutId = setTimeout(() => {  
      setShow(false); // Hide after 3 seconds  
    }, 3000);  
  
    return () => {  
      clearTimeout(timeoutId);  
    };  
  }, []);  
  
  return show ? <div>This will disappear in 3 seconds!</div> : null;  
}
```

6. Understanding Cleanup Functions

What is Cleanup?

A cleanup function is what you **return** from the effect function.

```
useEffect(() => {  
  // Setup code  
  const subscription = subscribeToService();  
  
  // Cleanup code (returned)  
  return () => {  
    subscription.unsubscribe();  
  };  
});
```

```
};  
}, []);
```

Think of it as:

- Effect function = **Setup**
- Cleanup function = **Teardown**

When Does Cleanup Run?

Two scenarios:

1. **Before effect runs again** (when dependencies change)
2. **When component unmounts** (removed from screen)

Cleanup Timeline Example

```
function Timer() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    console.log('🟢 Effect runs, count:', count);  
  
    const id = setInterval(() => {  
      console.log('🕒 Tick');  
    }, 1000);  
  
    return () => {  
      console.log('🔴 Cleanup runs, count was:', count);  
      clearInterval(id);  
    };  
  }, [count]); // Depends on count  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
    </div>  
  );  
}
```

```

    <button onClick={() => setCount(count + 1)}>Increment</button>
  </div>
);
}

```

Console output:

Initial render (count = 0):

● Effect runs, count: 0

🕒 Tick (every second)

Click button (count becomes 1):

● Cleanup runs, count was: 0 ← Old interval stopped

● Effect runs, count: 1 ← New interval started

🕒 Tick (every second)

Component unmounts:

● Cleanup runs, count was: 1 ← Interval stopped

Why Cleanup is Critical

Without cleanup:

```

useEffect(() => {
  setInterval(() => {
    console.log('Tick');
  }, 1000);
  // No cleanup!
}, []);

```

Problem:

Component mounts → Interval starts

Component unmounts → Interval STILL running (memory leak!)

With cleanup:

```
useEffect(() => {  
  const id = setInterval(() => {  
    console.log('Tick');  
  }, 1000);  
  
  return () => clearInterval(id);  
}, []);
```

Result:

Component mounts → Interval starts
Component unmounts → Cleanup runs → Interval stopped ✓

Common Things That Need Cleanup

1. Timers

```
useEffect(() => {  
  const id = setInterval(() => {}, 1000);  
  return () => clearInterval(id);  
}, []);
```

2. Event Listeners

```
useEffect(() => {  
  function handleScroll() {  
    console.log('Scrolling...');  
  }  
  
  window.addEventListener('scroll', handleScroll);  
  
  return () => {
```

```
window.removeEventListener('scroll', handleScroll);  
};  
, []);
```

3. Subscriptions

```
useEffect(() => {  
  const subscription = dataSource.subscribe();  
  
  return () => {  
    subscription.unsubscribe();  
  };  
, []);
```

7. Dependency Arrays - Complete Guide

The dependency array is the **second argument** to `useEffect`. It controls **when** the effect runs.

```
useEffect(() => {  
  // Effect code  
, [dependencies]); // ← This array
```

Three Patterns

Pattern 1: Empty Array `[]` - Run Once

```
useEffect(() => {  
  console.log('Runs once on mount');  
, []); // No dependencies
```

When it runs:

-  On component mount (first render)

- ❌ Never again (even if component re-renders)
- ✅ Cleanup only on unmount

Use for:

- Fetching data once
- Starting timers
- Subscribing to services

Example:

```
function App() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('/api/data')
      .then(res => res.json())
      .then(setData);
  }, []); // Fetch once on mount

  return <div>{data?.name}</div>;
}
```

Pattern 2: With Dependencies `[value]` - Run When Value Changes

```
useEffect(() => {
  console.log('Runs when count changes');
}, [count]); // Depends on count
```

When it runs:

- ✅ On component mount
- ✅ Whenever `count` changes
- ❌ When other state changes (not in array)
- ✅ Cleanup before each re-run and on unmount

Example:

```
function App() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    document.title = `Count: ${count}`;  
  }, [count]); // Update title when count changes  
  
  return (  
    <div>  
      <p>{count}</p>  
      <button onClick={() => setCount(count + 1)}>+</button>  
    </div>  
  );  
}
```

Pattern 3: No Array - Run After Every Render

```
useEffect(() => {  
  console.log('Runs after every render');  
}); // No dependency array at all
```

When it runs:

-  After every single render
-  Use very rarely (usually not needed)

Multiple Dependencies

```
useEffect(() => {  
  console.log('Runs when count OR name changes');  
}, [count, name]); // Multiple dependencies
```

Effect runs if ANY dependency changes.

Detailed Example: State NOT in Dependencies

```
function App() {  
  const [count, setCount] = useState(0);  
  const [name, setName] = useState('');  
  
  useEffect(() => {  
    console.log('🟢 Effect runs');  
  
    const intervalId = setInterval(() => {  
      console.log('🕒 Count is:', count);  
    }, 1000);  
  
    return () => {  
      console.log('🔴 Cleanup runs');  
      clearInterval(intervalId);  
    };  
  }, [count]); // Only depends on count, NOT name  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
      <p>Name: {name}</p>  
      <button onClick={() => setCount(count + 1)}>Count++</button>  
      <button onClick={() => setName(name + 'a')}>Name++</button>  
    </div>  
  );  
}
```

Timeline:

Initial **render** (count=0, name=''):

🟢 Effect runs

🕒 Count is: 0 (every second)

Click "Name++" (count=0, name='a'):

- Component re-renders
- useEffect checks: `[0]` vs `[0]` → SAME
- Effect SKIPPED ❌
- Old interval keeps running
- 🕒 Count is: 0 (still logs 0)

Click "Name++" again (count=0, name='aa'):

- Component re-renders
- useEffect checks: `[0]` vs `[0]` → SAME
- Effect SKIPPED ❌
- 🕒 Count is: 0

Click "Count++" (count=1, name='aa'):

- Component re-renders
- useEffect checks: `[0]` vs `[1]` → DIFFERENT ✅
- Cleanup runs (old interval cleared)
- Effect runs (new interval created)
- 🕒 Count is: 1 (now logs 1)

Key Point: Changing `name` does NOT trigger the effect because `name` is not in the dependency array.

Rules for Dependencies

Rule 1: If your effect uses a variable from the component, add it to dependencies.

```
function App() {  
  const [count, setCount] = useState(0);  
  
  useEffect(() => {  
    console.log(count); // Uses 'count'  
  }, [count]); // Must include 'count' in dependencies  
}
```

Rule 2: If you don't want the effect to re-run when something changes, don't include it.

```
function App() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('');

  useEffect(() => {
    // Only care about count, not name
    console.log(count);
  }, [count]); // Don't include 'name'
}
```

Rule 3: Empty array `[]` means "no dependencies, run once."

```
useEffect(() => {
  // Run once on mount
}, []);
```

8. Controlled vs Uncontrolled Inputs

Understanding Input Fields in React

Question: "Input fields work without React. Why use state?"

Answer: You need state to **control** what's in the input.

Uncontrolled Input (Browser Controls It)

```
function App() {
  return <input type="text" />;
}
```

Who controls the value? The **browser** (not React!)

What happens when you type:

1. You type "hello"
2. Browser updates the input
3. React does nothing

It works like plain HTML.

Problems with Uncontrolled Inputs

Problem 1: Can't Transform Input

Try to make input uppercase:

```
function App() {  
  const handleChange = (e) => {  
    console.log(e.target.value); // Can READ it  
    // But can't make it uppercase in the input!  
  };  
  
  return <input onChange={handleChange} />;  
}
```

Result: User types "hello" → Input shows "hello" (not "HELLO")

You can read the value but can't control what's displayed.

Problem 2: Can't Access Value Later

```
function App() {  
  const handleSubmit = () => {  
    // How do you get the input value here? 🤔  
    // You'd need refs or DOM queries  
  };  
  
  return (  
    <>  
    <input onChange={(e) => console.log(e.target.value)} />  
  )  
}
```

```

    <button onClick={handleSubmit}>Submit</button>
  </>
);
}

```

You can't access the input value in `handleSubmit` !

Problem 3: Can't Reset Input

```

function App() {
  const handleClear = () => {
    // How do you clear the input? 🤔
  };

  return (
    <>
      <input />
      <button onClick={handleClear}>Clear</button>
    </>
  );
}

```

Can't clear the input programmatically.

Controlled Input (React Controls It)

```

function App() {
  const [name, setName] = useState('');

  return (
    <input
      value={name} // ← React controls the value
      onChange={(e) => setName(e.target.value)}
    />
  );
}

```

```
);  
}
```

Who controls the value? React!

What happens when you type "R":

1. You press "R" key
2. onChange event fires
3. setName('R') called
4. name state becomes 'R'
5. Component re-renders
6. React sets input.value = 'R'
7. Browser displays 'R'

React overwrites the input value on every keystroke!

Controlled Input - Solutions to Problems

Solution 1: Transform Input

```
function App() {  
  const [name, setName] = useState('');  
  
  return (  
    <input  
      value={name}  
      onChange={(e) => setName(e.target.value.toUpperCase())}  
    />  
  );  
}
```

Result: User types "hello" → Input shows "HELLO" 

Solution 2: Access Value Anywhere

```

function App() {
  const [name, setName] = useState('');

  const handleSubmit = () => {
    console.log('Submitting:', name); // ✅ Easy access!
    fetch('/api/user', {
      method: 'POST',
      body: JSON.stringify({ name })
    });
  };

  return (
    <>
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <button onClick={handleSubmit}>Submit</button>
    </>
  );
}

```

Now you can use `name` anywhere in your component!

Solution 3: Reset Input

```

function App() {
  const [search, setSearch] = useState('');

  return (
    <>
      <input
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
    </>
  );
}

```



```

    <button onClick={() => setSearch('')}>Clear</button>
  </>
);
}

```

Click "Clear" → Input empties immediately! ✅

Solution 4: Validate While Typing

```

function App() {
  const [text, setText] = useState('');

  const handleChange = (e) => {
    const value = e.target.value;

    // Only allow max 10 characters
    if (value.length <= 10) {
      setText(value);
    }
    // If user tries to type 11th character, it's rejected!
  };

  return (
    <input
      value={text}
      onChange={handleChange}
    />
  );
}

```

User physically cannot type more than 10 characters! ✅

Solution 5: Use Value in Multiple Places

```
function App() {
  const [text, setText] = useState('');

  return (
    <>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <p>You typed: {text}</p>
      <p>Length: {text.length}</p>
      <p>Characters remaining: {280 - text.length}</p>
    </>
  );
}
```

Everything stays in sync! ✅

Visual Comparison

Uncontrolled (No `value` prop):

User types "hello"

↓

Browser updates input → Shows "hello"

↓

onChange fires → You get e.target.value = "hello"

↓

You can READ it, but that's it

You're a passenger. Browser drives.

Controlled (With `value` prop):

User types "h"
↓
onChange fires → e.target.value = "h"
↓
setState("h")
↓
Component re-renders
↓
React sets input.value = "h"
↓
Browser shows "h"

You're the driver. You control everything.

The Rule

To make an input controlled:

```
<input  
  value={stateVariable}      // 1. value prop  
  onChange={(e) ⇒ setState(...)} // 2. onChange handler  
>
```

Both are required!

- Without `value`: Browser controls it (uncontrolled)
 - Without `onChange`: Input is frozen (can't type)
-

9. Practice Examples

Example 1: Fetch Users from GitHub

```
import { useState, useEffect } from 'react';  
  
function UserList() {
```

```

const [users, setUsers] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  async function fetchUsers() {
    setLoading(true);
    const response = await fetch('https://api.github.com/users?per_page=10');
    const data = await response.json();
    setUsers(data);
    setLoading(false);
  }

  fetchUsers();
}, []); // Fetch once on mount

if (loading) return <div>Loading...</div>;

return (
  <ul>
    {users.map(user => (
      <li key={user.id}>
        <img src={user.avatar_url} width="50" height="50" />
        <span>{user.login}</span>
      </li>
    ))}
  </ul>
);
}

```

Example 2: Search with Button Click

```

import { useState, useEffect } from 'react';

function SearchUsers() {

```

```

const [users, setUsers] = useState([]);
const [count, setCount] = useState(10);
const [shouldFetch, setShouldFetch] = useState(false);
const [loading, setLoading] = useState(false);

useEffect(() => {
  if (!shouldFetch) return; // Skip if button not clicked

  async function fetchUsers() {
    setLoading(true);
    const response = await fetch(
      `https://api.github.com/users?per_page=${count}`
    );
    const data = await response.json();
    setUsers(data);
    setLoading(false);
    setShouldFetch(false); // Reset for next search
  }

  fetchUsers();
}, [shouldFetch]);

return (
  <>
    <h1>GitHub Users</h1>
    <input
      type="number"
      value={count}
      onChange={(e) => setCount(Number(e.target.value))}
      min="1"
      max="100"
    />
    <button onClick={() => setShouldFetch(true)}>Search</button>

    {loading && <p>Loading...</p>}
  </>
)

```

```

    <ul>
      {users.map(user => (
        <li key={user.id}>{user.login}</li>
      ))}
    </ul>
  </>
);
}

```

Example 3: Digital Clock

```

import { useState, useEffect } from 'react';

function Clock() {
  const [time, setTime] = useState(new Date().toLocaleTimeString());
  const [showTime, setShowTime] = useState(true);

  useEffect(() => {
    if (!showTime) return; // Don't run interval if hidden

    const intervalId = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);

    return () => clearInterval(intervalId);
  }, [showTime]);

  return (
    <>
      <button onClick={() => setShowTime(!showTime)}>
        {showTime ? 'Hide' : 'Show'} Time
      </button>
      {showTime && <h1>Time: {time}</h1>}
    </>
  )
}

```

```
);  
}
```

Example 4: Countdown Timer

```
import { useState, useEffect } from 'react';  
  
function Countdown() {  
  const [seconds, setSeconds] = useState(60);  
  const [isRunning, setIsRunning] = useState(false);  
  
  useEffect(() => {  
    if (!isRunning || seconds === 0) return;  
  
    const intervalId = setInterval(() => {  
      setSeconds(prev => prev - 1);  
    }, 1000);  
  
    return () => clearInterval(intervalId);  
  }, [isRunning, seconds]);  
  
  return (  
    <>  
      <h1>Time: {seconds}s</h1>  
      <button onClick={() => setIsRunning(!isRunning)}>  
        {isRunning ? 'Pause' : 'Start'}  
      </button>  
      <button onClick={() => setSeconds(60)}>Reset</button>  
    </>  
  );  
}
```

Example 5: Search with Auto-save (Debouncing)

```
import { useState, useEffect } from 'react';

function AutoSave() {
  const [text, setText] = useState('');
  const [saved, setSaved] = useState('');

  useEffect(() => {
    // Wait 1 second after user stops typing
    const timeoutId = setTimeout(() => {
      console.log('Saving:', text);
      setSaved(text); // Simulate saving
    }, 1000);

    return () => clearTimeout(timeoutId);
  }, [text]);

  return (
    <>
      <textarea
        value={text}
        onChange={(e) => setText(e.target.value)}
        rows="5"
        cols="50"
      />
      <p>Last saved: {saved}</p>
    </>
  );
}
```

10. Common Mistakes to Avoid

Mistake 1: Calling setState During Render (Infinite Loop)


```
// ❌ Wrong
function App() {
  const [count, setCount] = useState(0);

  setCount(count + 1); // Called during render!

  return <div>{count}</div>;
}
```

Result: Infinite loop! Component keeps re-rendering.

Fix: Use `useEffect` or event handler

```
// ✅ Correct
function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    setCount(count + 1);
  }, []); // Or in a button click handler

  return <div>{count}</div>;
}
```

Mistake 2: Missing Cleanup (Memory Leak)

```
// ❌ Wrong
useEffect(() => {
  setInterval(() => {
    console.log('Tick');
  }, 1000);
  // No cleanup!
}, []);
```

Problem: Interval keeps running after component unmounts.

Fix: Always cleanup timers

```
// ✅ Correct
useEffect(() => {
  const id = setInterval(() => {
    console.log('Tick');
  }, 1000);

  return () => clearInterval(id);
}, []);
```

Mistake 3: Making Effect Function async

```
// ❌ Wrong
useEffect(async () => {
  const data = await fetch('/api');
}, []);
```

Problem: Can't make effect function async!

Fix: Create async function inside

```
// ✅ Correct
useEffect(() => {
  async function fetchData() {
    const data = await fetch('/api');
  }

  fetchData();
}, []);
```

Mistake 4: Missing Dependencies

```
// ❌ Wrong
function App() {
```

```
const [count, setCount] = useState(0);

useEffect(() => {
  console.log(count); // Uses 'count'
}, []); // But doesn't include it!
}
```

Problem: Effect won't re-run when `count` changes. You'll see stale values.

Fix: Include all used variables

```
// ✅ Correct
useEffect(() => {
  console.log(count);
}, [count]); // Include 'count'
```

Mistake 5: Unnecessary Dependencies

```
// ❌ Wrong
function App() {
  const [trigger, setTrigger] = useState(0);
  const [number, setNumber] = useState(10);

  useEffect(() => {
    fetch(`/api?count=${number}`);
  }, [trigger, number]); // Includes 'number' unnecessarily
}
```

Problem: If you only want to fetch on button click (not when `number` changes), don't include `number`.

Fix: Only include what should trigger the effect

```
// ✅ Correct
useEffect(() => {
```

```
fetch(`/api?count=${number}`);  
}, [trigger]); // Only trigger, not number
```

Mistake 6: Frozen Input (Missing onChange)

```
// ❌ Wrong  
function App() {  
  const [text, setText] = useState('');  
  
  return <input value={text} />; // No onChange!  
}
```

Problem: Can't type in the input! It's frozen.

Fix: Always have onChange with value

```
// ✅ Correct  
function App() {  
  const [text, setText] = useState('');  
  
  return (  
    <input  
      value={text}  
      onChange={(e) => setText(e.target.value)}  
    />  
  );  
}
```

Summary Cheatsheet

useEffect Patterns

```
// Run once on mount  
useEffect(() => {
```

```

// Fetch data, start timer, subscribe
return () => {
  // Cleanup on unmount
};
}, []);

// Run when dependency changes
useEffect(() => {
  // Update based on state/prop
  return () => {
    // Cleanup before re-run
  };
}, [dependency]);

// Run after every render (rare)
useEffect(() => {
  // Runs every time
});

```

Controlled Input Pattern

```

const [value, setValue] = useState('');

<input
  value={value}
  onChange={(e) => setValue(e.target.value)}
/>

```

Key Points to Remember

1. **useEffect runs AFTER render** - Never blocks the UI
2. **Always cleanup timers and listeners** - Prevent memory leaks
3. **Dependencies control when effect runs** - `[]` = once, `[dep]` = when dep changes

4. **Can't make effect async** - Create async function inside instead
5. **Controlled inputs need value + onChange** - Both are required
6. **Empty array `[]` means run once** - Not "no dependencies needed"

End of Notes

Quick Reference

When to use useEffect?

-  Fetching data from API
-  Starting/stopping timers
-  Subscribing to services
-  Adding/removing event listeners
-  Updating document title
-  Calculating values from state (just use variables)
-  Handling button clicks (use event handlers)

Cleanup is needed for:

-  `setInterval` / `setTimeout`
-  Event listeners (`addEventListener`)
-  Subscriptions
-  WebSocket connections
-  Regular `fetch` calls (they clean up automatically)
-  Setting state (doesn't need cleanup)